

Arkitektur principper i praksis

M9.02 - Monitorering med Prometheus

Formål

Denne opgave etablerer et monitorerings miljø for performance testing i jeres Minikube kubernetes-klynge og indeholder en simple microservice som er instrumenteret med simple metrikker.

Formålet med opgaven er at forstå infrastrukturen og værktøjerne for opsamling af metrikker og logdata, til brug i jeres egen løsning.

Forudsætninger

- Denne opgave kræver at du har en Kubernetes klynge oppe at køre, f.eks. i form af Minikube eller lign.
- Det kræves også at du har installeret en Grafana-infrastruktur, f.eks. i form af `helm`-chartet `grafana/loki-stack` (se opgave **M8.03**)

k8s-performance

Denne opgave er baseret på et demo-projekt som du finder på <https://github.com/HNRK-JNSN/k8s-performance.git>, som du bør hente og tilpasse for at løse opgaverne herunder.

Opgave A - Hent demo-projektet

1. Lav en folder på din lokale maskine og hent demo projektet fra flg. `GitHub`-adresse:

`https://github.com/HNRK-JNSN/k8s-performance.git`

2. Tag et øjeblik og dan dig et overblik over folder-strukturen og hvilke filer er med i projektet. Læs f.eks. `Readme.md` filen i roden af projektet.

Opgave B - Prometheus.yaml konfiguration

Du skal nu udvide din Minikube-klynge med Prometheus og tilknyttede hjælpeapplikationer vha. af følgende instrukser.

1. Start med at opdatere dit lokale Helm-repo:

```
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-chart:  
$ helm repo update
```

2. Fra en terminal, gå ind i folderen Minikub-Env, og kørs en Helm -installation af kube-prometheus-stack vha. flg. kommando:

```
$ helm install kube-prometheus-stack \  
    prometheus-community/kube-prometheus-stack \  
    --namespace monitoring --values prometheus.yaml \  
    --create-namespace
```



Bemærk

Filen `prometheus.yaml`, som indeholder tilpasningsværdierne er allerede tilpasset, men tag et kig i den alligevel, hvis du ønsker at ændre din opsætning senere.

3. Tjek at der er kommet nye services i `monitoring`-namespace:

```
$ kubectl get services -n monitoring
```

Opgave C - Kig i koden

Du behøver ikke at rette i koden for at afprøve din løsning, men tag alligevel et kig i C#-koden som du finder i folderen `TestService`.

Kig på :

- **TestService.csproj:**
 - Hvilke `nugets` er brugt
- **Program.cs:**
 - Hvordan aktiveres OpenTelemetry til at anvende Prometheus
 - Hvilke metrikker anvendes (se `AddMeter` og `AddView`)
 - Hvilke endepunkter kan kaldes
- **NLog.config:**
 - Hvilke `targets` er aktiveret
 - Hvordan finder NLog `Loki` -endepunktet

Opgave D - Byg TestService docker image

Den nemmeste måde at bygge `TestService`'en på er at gå ind i `staging` -folderen og tilpasse `docker-compose.yml`. Du behøver kun at omdøbe `image` -navnet til at passe til din egen DockerHub-konto.

Fra terminalen kan du herefter køre kommandoerne:

```
$ docker compose build
$ docker compose push
```

Opgave E - Deploy din TestService i klyngen

Du har nu et `Docker image` i DockerHub som er instrumenteret til at logge både til Loki og til Prometheus. Det næste trin er at få oprettet en Kubernetes Pod med respektiv Service.

Gå tilbage ind i Minikube-Env og tag et kig i filen `weatherforecast-svc.yaml`. Den indeholder instrukser til deployment af 3 Kubernetes objekter: en `Pod`, en `Service` og en `Service Monitor`.

Tilpas navnet på imaget som skal køres når der oprettes en container i din Pod -deployment specifikation.

Hvad er en Service Monitor?

En Kubernetes Service Monitor er en specialiseret ressource, der bruges til at konfigurere overvågning af tjenester i et Kubernetes-cluster ved hjælp af Prometheus.

Den linker din TestService med Prometheus således at Prometheus ikke skal konfigureres manuelt.

Når filen er tilpasset så lav et deployment med:

```
$ kubectl apply -f weatherforecast-svc.yaml
```

Genstart Prometheus Pod for at fremtvinge anvendelse af Service Monitor :

```
$ kubectl rollout restart deployments -n monitoring
```

Opgave F - Test servicen

1. Du skal bruge en tunnel ind i servicen for weatherforecast , så køр følgende kommando:

```
$ minikube service weatherforecast-service -n default --url
```

2. Afprøv følgende endepunkter med curl eller brug en browser:

```
$ export WeatherforecastURL=<URL_FRA_PUNKT_1>
$ curl $WeatherforecastURL
$ curl $WeatherforecastURL/weatherforecast
$ curl $WeatherforecastURL/telemetry
$ curl $WeatherforecastURL/version
$ curl $WeatherforecastURL/metrics
```

? metrics endepunktet

metrics -endepunktet er ikke defineret i TestService-projektet. Hvor kommer det så endepunkt fra?

Opgave F - Forbind Grafana og Prometheus

Tilgå din Grafana portal (kig i **opgave M8.03** for hvordan du finder bruger og password og opretter en tunnel).

Tilføj en ny datakilde af typen Prometheus .

? Prometheus URL

Grafana skal bruge Prometheus interne URL for at etablere en forbindelse. Hvordan finder du den?

Når datakilden er forbundet med succes så anvend Explore data -knappen (øverst) til at finde nogle metrikker fra din TestService. Brug gerne Metrics explorer til at navigere i dine muligheder.

Opgave G - Grafana Eksempel Dashboard

Du kan importere et færdigt lavet dashboard ind i Grafana fra deres [dashboard-repo](#). Find f.eks. Id 'et på [ASP.NET Core](#) og lav et nyt dashboard hvor du vælger  import dashboard .

Note: Tilføj også ASP.NET Core Endpoint (id = 19925) hvis du har valgt 19924. De linker sammen.

✓ Opgave H - Performance Test

Fra folderen `Minikube-Env` kan du køre scriptet `load-test.sh` som kalder dine endepunkter et antal gange. Husk at tilpasse URL, så den passer til din lokale adresse.

Find herefter dine data i Grafana og lav et nyt dashboard med en graf af dit histogram. Hint: Kig i **Program.cs** under `builder.AddView()` for hvad du skal søge efter.

